

Defesa da Fase B — Camada de Ligação de Dados

1. Visão Geral

O nosso sistema implementa uma Pilha Protocolar (Stack) personalizada. A Camada 2 junta dois canais físicos unidirecionais (ótico e elétrico) num canal lógico bidirecional (Full-Duplex). Para garantir fiabilidade num meio ruidoso, abdicámos de velocidade bruta em favor de resiliência, implementando uma arquitetura HARQ (Hybrid Automatic Repeat Request), que combina correção proativa no canal (FEC) com retransmissão reativa (Stop-and-Wait).

2. Descrição dos módulos

- **Camada de Aplicação (Interfaces):**
 - `app_cliente.py` - Injeta os ficheiros e fragmenta os dados.
 - `app_servidor.py` - Fica à escuta, guarda o ficheiro no disco e gere o timeout de sessão.
- **Core da Camada de Ligação:**
 - `camada_ligacao_dados.py` - Integra todos os módulos abaixo e expõe a API.
- **Módulos de Processamento de Rede:**
 - `camada_fisica.py` - Interface UART (pyserial) que gere a Thread de receção (Full-Duplex).
 - `framing_hdlc.py` - Algoritmo de Byte Stuffing/Destuffing e flags SOF.
 - `encapsulamento.py` - Estruturação C-style do Cabeçalho (MACs, Type, Seq, Length).
 - `crc8.py` - Matemática da divisão polinomial (FCS) para deteção de erros.
 - `fec_hamming.py` - Matemática do Código de Hamming (7,4) para correção no ar.
 - `maquina_estados_arq.py` - Lógica Stop-and-Wait (alternância do bit de sequência 0/1).
 - `temporizador_arq.py` - Gestor de Timeouts e retransmissões.
- **Utilitários:** `logger.py` (Módulo visual do terminal) e `constantes_protocolo.py`.

3. Conceitos essenciais

- **Modo Raw:** É ler e escrever os *bytes* em bruto na porta série. O sistema operativo não mexe, não formata nem interpreta os dados.
- **Baud Rate (9600 bps):** A velocidade física do cabo. A 9600 bps, cada bit demora cerca de 104 microssegundos a ser transmitido.
- **Full-Duplex:** Capacidade de falar e ouvir em simultâneo. No nosso software, o recetor corre numa *Thread* separada (em *background*), permitindo receber dados sem bloquear o emissor.

- **Stop-and-Wait:** Protocolo de controlo de fluxo onde o emissor envia uma trama e fica bloqueado à espera da confirmação (ACK) antes de enviar a seguinte.
- **CRC-8:** Algoritmo matemático de divisão polinomial. Deteta erros de transmissão com quase 100% de precisão (muito superior ao *Checksum*, que falha se os bytes trocarem de ordem).
- **Byte Stuffing:** Ocultar a *flag* de fim de trama (0x7E) se ela aparecer "sem querer" no meio da foto ou do texto, evitando que o recetor corte a mensagem a meio.
- **Throughput:** A velocidade real e útil. É o que sobra dos 9600 bps depois de pagarmos as "taxas" do protocolo.
- **Overhead:** O "imposto" da rede. São os *bytes* extra que não pertencem ao ficheiro (Cabeçalhos, ACKs, bits de paridade de Hamming) e os tempos mortos.

4. Explicação da trama

O formato da trama foi desenhado para garantir a delimitação e o transporte da informação. A delimitação é feita através da flag 0x7E (SOF - Start of Frame) e para garantir a transparência dos dados (evitando que um byte 0x7E no payload seja interpretado como o fim da trama), o sistema aplica o algoritmo de Byte Stuffing.

SOF (1byte) 0x7E	Destination MAC (1 byte) 0x02	Source MAC (1 byte) 0x01	TYPE (1 byte) Ex: 0x01	SEQ_NUM (1 byte) 0 ou 1	Length (1 byte) Tamanho real	PAYLOAD 1 a 256 Bytes Dados + Hamming	FCS (1 Byte) Pol: 0x07	SOF (1byte) 0x7E
------------------------	--	---------------------------------------	----------------------------------	-----------------------------------	---	--	----------------------------------	------------------------

5. Estratégia para a defesa oral

Instrução 1: Explicar o Tempo Total na Consola - (O que fazer): Mostra a consola após a transferência. Exemplo: Imagem demorou 12.91s (Teórico: 12.23s). (O que dizer):

"Professor, como pode observar, a consola regista 12.91s de tempo prático, face aos 12.23s do nosso modelo teórico. Este desvio inferior a 6% valida o nosso planeamento, mas revela as entropias de um sistema real. O tempo teórico reflete a serialização física (960 Bytes/segundo a 9600 bps). No entanto, o tempo prático dilata devido a três fatores que são: 1º imprevisibilidade do Framing - o nosso algoritmo de Byte Stuffing é dinâmico, ou seja se a matriz de pixéis da imagem contiver demasiados bytes coincidentes com 0x7E ou 0x7D, a injeção de bytes faz a trama 'aumentar' fisicamente no cabo. 2º Latência Computacional, ou seja, a encriptação matemática do Hamming (7,4) e a divisão do CRC-8 exigem um grande processamento em Python (Context Switching e GIL), introduzindo atrasos de milissegundos por trama. 3º Introduzimos sleeps propositados (0.01s) na receção para evitar que o poder de processamento do

PC asfixiasse o buffer limitado da UART na ESP32. O desvio é, portanto, o custo pela estabilidade."

Instrução 2: Demonstração do Link Misto (Ótico + Elétrico) - (O que fazer): Aponta para a montagem física na bancada. PC1 (TX) ligado ao LED. PC2 (RX) ligado ao Fotodetector. O TX do PC2 volta ao RX do PC1 por um fio de cobre. O GND é partilhado. (O que dizer):

"A nossa implementação de software é independente em relação ao meio físico, ou seja, o plano de dados (Data Plane) que carrega as tramas de texto e imagem de 208 bytes está a ser injetado exclusivamente pelo canal ótico (Simplex). Por outro lado, o plano de controlo (Control Plane) que transporta apenas os ACKs de 8 bytes regressa pelo canal elétrico de cobre (Simplex em sentido inverso). A nossa Camada de Ligação, é suportada por Multithreading, independente da infraestrutura física e entrega à Aplicação um serviço lógico adequado, sem erros e Full-Duplex."

Instrução 3: Justificar a Taxa de Transferência (Porquê "tão lento"?) - (O que fazer se o professor disser que outros grupos enviaram mais rápido): Sorrir e usar o trunfo do HARQ. (O que dizer):

"O nosso débito real (Throughput) é inferior ao de um grupo com uma implementação básica porque nós não construímos um ARQ simples, nós construímos um sistema HARQ (Hybrid-ARQ). O nosso payload sofre uma perda de 100% de overhead (dados úteis de 100 bytes transformam-se em 200 bytes no canal) devido à aplicação do Código de Hamming (7,4) antes do encapsulamento. Foi uma escolha consciente pois trocámos largura de banda por fiabilidade num canal com bastante ruído. O nosso sistema é capaz de detetar e corrigir erros de bit único 'no ar', reconstruindo o pacote no recetor sem acionar a máquina de estados para retransmissão. Num ambiente de aeroporto com interferência luminosa transitória, o custo do nosso FEC é muito menor do que o colapso contínuo provocado por Timeouts sucessivos de mais de 2 segundos num protocolo Stop-and-Wait padrão."

7. Demonstração de acordo com o pedido

-Deverão demonstrar a transferência fiável de um conjunto de tramas e apresentar o tempo total da transferência na consola.

- Deverão explicar o tempo total da transferência em função da taxa de transmissão usada na ligação física e do tamanho das tramas.

- Idealmente, deverão demonstrar o funcionamento do sistema usando a ligação ótica num sentido e a ligação elétrica no outro sentido.

1. Demonstração da transferência fiável e tempo total na consola

- **Onde está:** No ficheiro `app_cliente.py` e nos **Logs da consola**.

- **Prova no Código:** * A fiabilidade é garantida pela classe `CamadaDeLigacao` em `camada_ligacao_dados.py`, que utiliza o **ARQ Stop-and-Wait** e o **FEC (Hamming)** para assegurar que nenhum bit se perde.
 - O cálculo do tempo está nas linhas 41 a 51 do `app_cliente.py`. O programa inicia um cronómetro (`time.time()`) antes do envio e subtrai o valor final após a última trama.
- **Prova na Consola:** Quando executas o emissor, a última linha diz: `[SUCESSO]` Ficheiro totalmente enviado em X.XX segundos!.

2. Explicação do tempo total (Taxa física vs. Tamanho das tramas)

- **Onde está:** No documento de **Especificação** (Capítulo 4) que preparámos para entrega.
- **A lógica técnica:** * **Taxa Física:** Explicas que a 9600 bps, devido aos bits de *start/stop*, a taxa real é de **960 Bytes/s**.
 - **Tamanho das Tramas:** Explicas que cada bloco de 100 bytes úteis duplica para **200 bytes** devido ao Hamming (FEC). Adicionando os 8 bytes de overhead (Cabeçalho, FCS, SOFs), tens tramas físicas de **208 bytes**.

Cálculo: O tempo total é a soma do tempo de transmissão dessas tramas de 208 bytes + o tempo dos ACKs de 8 bytes + os *sleeps* de processamento. $T_{ciclo} = 0,2166 + 0,0083 + 0,0100 = 0.2349 \text{ segundos}$

3. Ligação Ótica num sentido e Elétrica no outro

- **Onde está:** Na **Montagem Física (Hardware)** e na arquitetura **Full-Duplex** do software.
- **Prova no Software:** No ficheiro `camada_fisica.py`, a utilização de *Threads* permite que o sistema envie dados (pelo TX/LED) e receba ACKs (pelo RX/Fio) ao mesmo tempo, sem que um bloqueie o outro.
- **Prova no Hardware:** Tens de garantir na bancada que:
 - **Sentido Emissor -> Recetor:** O cabo de dados passa pelo circuito do LED e viaja pelo ar até ao Fototransístor.
 - **Sentido Recetor -> Emissor (Retorno):** Um fio de cobre liga o TX da ESP32 recetora diretamente ao RX da ESP32 emissora para trazer os ACKs de confirmação.

Pergunta:

- Onde é que garantem a fiabilidade?

A fiabilidade é garantida na `camada_ligacao_dados.py` porque utilizamos o código de Hamming para corrigir erros no ar e o protocolo ARQ para retransmitir se o feixe for cortado. O tempo total que vê na consola é calculado pela aplicação cliente, e reflete o overhead de codificação de tramas de 208 bytes a uma taxa física de 960 bytes por segundo.

6. Perguntas e Respostas

P1: O vosso cálculo de Throughput é muito baixo comparado com os 9600 bps nominais.

Porquê? - O nosso tempo reflete o uso de HARQ. O grupo decidiu ir além do básico e implementar o Código de Hamming (7,4). Isto significa que o nosso *payload* sofre um *overhead* de 100% (100 bytes úteis tornam-se 200 bytes no ar). Trocámos velocidade bruta por fiabilidade extrema. Num ambiente com ruído luminoso, corrigir bits no ar é mais eficiente do que esperar 2 segundos por um *timeout* repetidas vezes.

P2: O enunciado exige comunicação Full-Duplex. Como provam isso se o canal de luz só vai num sentido?

- O Full-Duplex é uma propriedade da nossa arquitetura de *software*. Enquanto o Emissor está bloqueado no temporizador à espera do ACK, a nossa *Thread* de receção (camada_fisica.py) continua viva a ler o RX. Isto prova que o sistema consegue escutar e transmitir autonomamente em simultâneo.

P3: Como é que o sistema resolve o "Problema do ACK Perdido"?

- Através da alternância do campo *Sequence Number* (0 e 1). Se o ACK se perder no fio de cobre, o emissor dá *timeout* e retransmite a trama. O recetor vê chegar uma trama com um número de sequência repetido. Ele percebe o erro, rejeita guardar os dados em duplicado, mas reenvia o ACK para destrancar o emissor.

P4: Qual era a razão do limite mínimo de 68 bytes no vosso código inicial?

- Foi um artefacto teórico que herdámos do estudo da norma Ethernet (CSMA/CD), onde o tamanho mínimo da trama garante a deteção de colisões. Como o nosso link é ponto-a-ponto, bidirecional e não tem contenção de meio físico, esse limite não faz sentido e a nossa API já aceita *payloads* a partir de 1 byte.

P5: Porque é necessário byte stuffing se já existe o campo Length?

- Porque o campo Length também pode ser corrompido, e o stuffing preserva a capacidade de reencontrar delimitadores válidos mesmo quando o conteúdo da trama contém bytes reservados.

P6: Porque é que o Stop-and-Wait é suficiente neste projeto?

- Porque o contexto experimental é simples e controlado. O objetivo principal é garantir fiabilidade, e não maximizar a eficiência de uma rede de longa distância.

P8: O CRC corrige erros?

- Não. O CRC apenas deteta erros. A recuperação é feita pela retransmissão.

P9: O que é overhead?

- É o custo protocolar adicional que não corresponde aos dados úteis do ficheiro.

P9: O que acontece se o payload tiver 0x7E ou 0x7D?

- Esses bytes têm de ser escapados pelo mecanismo de byte stuffing para não serem confundidos com delimitadores.

P10: Qual é a diferença entre bps e bytes por segundo?

- Bps mede bits por segundo; bytes por segundo mede unidades de 8 bits por segundo. São grandezas diferentes.

P11: O que é backoff? - É um atraso intencional entre tentativas de envio, usado para evitar agressividade excessiva na retransmissão ou para dar margem ao processamento do recetor.

P12: Quantas mensagens cabem num ficheiro de 1 KB? - Depende do tamanho útil do payload por trama. Se cada trama transportar 64 bytes úteis, um ficheiro de 1024 bytes corresponde a cerca de 16 tramas de dados, excluindo overhead e possíveis retransmissões.

P13: Qual é a capacidade real a 9600 bps? - A taxa nominal é 9600 bps, mas a taxa útil real é menor devido ao framing, ao stuffing, ao CRC, aos ACKs e às esperas por confirmação.

P14: O que acontece se a comunicação for interrompida? - O emissor atinge timeout, retransmite e retoma a comunicação no ponto em que parou quando o canal volta a ficar disponível.

ANEXO B — Mapeamento do Enunciado e Comentários do Código

Requisito do Enunciado	Como foi implementado	Localização no Código
Porta Série em Modo Raw e Parâmetros	Inicialização via pyserial sem controlo de hardware, a 9600 bps.	camada_fisica.py (Classe Camada2_FullDuplex)
Operação Full-Duplex	Ciclo de leitura da porta a correr numa Thread tipo Daemon separada do fluxo de envio.	camada_fisica.py (Método _receber_continuamente)
Delimitação de Tramas (LLC)	Uso da flag delimitadora SOF (0x7E) com a proteção automática contra erros de código	framing_hdlc.py (Funções byte_stuffing e destuffing)
Controlo de Erros (Deteção)	Cálculo do Frame Check Sequence (FCS) gerado via algoritmo de divisão polinomial.	crc8.py (Módulo de verificação)
Controlo de Erros (Correção - Extra)	Implementação de FEC (Forward Error Correction) com inserção de redundância matemática.	fec_hamming.py (Código de Hamming 7,4)
Controlo de Fluxo e ARQ	Arquitetura de alternância de bits que gera ACKs/NACKs e descarta tramas duplicadas.	maquina_estados_arq.py (Protocolo Stop-and-Wait)
Timeout e Recuperação no Ponto	O bloqueio esgota ao fim de 2.0s, retransmitindo a trama perdida até 7 vezes.	temporizador_arq.py (Classe GestorTimeout)
Conceito de API Abstraída	A Camada 7 chama uma função simples e a Camada 2 isola todo o mecanismo de rede subjacente.	camada_ligacao_dados.py (Primitiva enviar_ficheiro(payload, tipo))